

COURSE CODE/TITLE:	CPE 212	SECTION:	CPE31S2
DATE PERFORMED:	Aug 25, 2026	DATE SUBMITTED:	Aug 25, 2026
NAME:	Loterte, Mark David M.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 4 Running Elevated Ad hoc Commands

1. PROCEDURE OUTPUT

Task 1: Run elevated ad hoc commands

Locally, we use the command `sudo apt update` when we want to download package information from all configured resources. The sources are often defined in `/etc/apt/sources.list` file and other files located in `/etc/apt/sources.list.d/` directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

```
ansible all -m apt -a update_cache=true
```

What is the result of the command? Is it successful?

```
mdmlot@workstation:~/ansible-lab$ ansible all -m apt -a update_cache=true
192.168.56.105 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}
192.168.56.104 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
/var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
}
mdmlot@workstation:~/ansible-lab$
```

The command failed

Try editing the command and add something that would elevate the privilege. Issue the command `ansible all -m apt -a update_cache=true --become --ask-become-pass`. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The `update_cache=true` is the same thing as running `sudo apt update`. The `--become` command elevates the privileges and the `--ask-become-pass` asks for the password. For now, even if we only changed the packaged index, we were able to change

something on the remote server. You may notice after the second command was executed, the status is CHANGED compared to the first command, which is FAILED.

```
mdmlot@workstation:~/ansible-lab$ ansible all -m apt -a update_cache=true --become
me --ask-become-pass
BECOME password:
192.168.56.104 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787662975,
  "cache_updated": true,
  "changed": true
}

192.168.56.103 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "cmd": "apt-get update",
  "msg": "[Errno 2] No such file or directory: b'apt-get'",
  "rc": 2,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "",
  "stdout_lines": []
}
```

Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: `ansible all -m apt -a name=vim-nox --become --ask-become-pass`. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
mdmlot@workstation:~/ansible-lab$ ansible all -m apt -a name=vim-nox --become --ask-become-pass
BECOME password:
192.168.56.104 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787662975,
  "cache_updated": false,
  "changed": true,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "Reading package lists...\nBuilding dependency tree...\nReading state information...\nThe following packages were automatically installed and are no longer required:\n libfwupd2 libwoff1\nUse 'sudo apt autoremove' to remove them.\nThe following additional packages will be installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n rake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc\n ruby3.2 rubygems-integration vim-common vim-runtime vim-tiny xxd\nSuggested packages:\n ri ruby-dev bundler cscope vim-doc indent\nThe following NEW packages will be installed:\n fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby libruby3.2\n rake ruby ruby-net-telnet ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc\n ruby3.2 r
```

a. Verify that you have installed the package in the remote servers. Issue the command which vim and the command apt search vim-nox respectively. Was the command successful?

```
mdmlot@server1:~$ which vim
/usr/bin/vim
mdmlot@server1:~$ apt search vim-nox
Sorting... Done
Full Text Search... Done
vim-nox/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed]
Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/noble-updates,noble-security,now 2:9.1.0016-1ubuntu7.19 amd64 [installed,automatic]
Vi IMproved - enhanced vi editor - compact version

mdmlot@server1:~$
```

```
[mdmlot@server2 ~]$ which vim
/usr/bin/vim
[mdmlot@server2 ~]$ dnf search vim-nox
CentOS Stream 9 - BaseOS                1.2 MB/s | 9.0 MB    00:07
CentOS Stream 9 - AppStream             5.7 MB/s | 29 MB    00:04
CentOS Stream 9 - Extras packages       3.1 kB/s | 22 kB    00:07
Last metadata expiration check: 0:00:01 ago on Tue 25 Aug 2026 09:08:31 PM PST.
No matches found.
[mdmlot@server2 ~]$ dnf search vim
Last metadata expiration check: 0:00:20 ago on Tue 25 Aug 2026 09:08:31 PM PST.
===== Name & Summary Matched: vim =====
vim-X11.x86_64 : The VIM version of the vi editor for the X Window System - GVim
vim-common.x86_64 : The common files needed by any version of the VIM editor
vim-enhanced.x86_64 : A version of the VIM editor which includes recent
                    : enhancements
vim-filesystem.noarch : VIM filesystem layout
vim-minimal.x86_64 : A minimal version of the VIM editor
[mdmlot@server2 ~]$ S
```

The installation command was successful for both Ubuntu and CentOS servers.

b. Check the logs in the servers using the following commands: `cd /var/log`. After this, issue the command `ls`, go to the folder `apt` and open `history.log`. Describe what you see in the `history.log`.

```
Remove: linux-hwe-6.17-headers-6.17.0-14:amd64 (6.17.0-14.14~24.04.1)
End-Date: 2026-08-24 19:53:53

Start-Date: 2026-08-24 19:53:56
Commandline: /usr/bin/unattended-upgrade
Remove: linux-tools-6.17.0-14-generic:amd64 (6.17.0-14.14~24.04.1)
End-Date: 2026-08-24 19:53:57

Start-Date: 2026-08-24 19:54:00
Commandline: /usr/bin/unattended-upgrade
Remove: linux-hwe-6.17-tools-6.17.0-14:amd64 (6.17.0-14.14~24.04.1)
End-Date: 2026-08-24 19:54:00

Start-Date: 2026-08-25 21:05:10
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Op>
Requested-By: mdmlot (1000)
Install: ruby-sdbm:amd64 (1.0.0-5build4, automatic), liblua5.1-0:amd64 (5.1.5-9>
Upgrade: xxd:amd64 (2:9.1.0016-1ubuntu7.18, 2:9.1.0016-1ubuntu7.19), vim-common>
End-Date: 2026-08-25 21:06:05
█

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

At the very bottom, I can see the vim installation log, along with the upgrade commands ran yesterday before the

actual activity.

This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

a. Issue the command: `ansible all -m apt -a name=snapd --become --ask-become-pass`

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers?

```
mdmlot@workstation:~/ansible-lab$ ansible all -m apt -a name=snapd --become --ask-become-pass
BECOME password:
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
192.168.56.103 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "cmd": "apt-get update",
  "msg": "[Errno 2] No such file or directory: b'apt-get'",
  "rc": 2,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "",
  "stdout_lines": []
}
192.168.56.104 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787664051,
  "cache_updated": false,
  "changed": false
}
mdmlot@workstation:~/ansible-lab$
```

The command was successful only for Ubuntu because CentOS uses dnf. Nothing was changed with the Ubuntu Server.

b. Now, try to issue this command: `ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass`

```
mdm1ot@workstation:~/ansible-lab$ ansible all -m apt -a "name=snapd state=latest"
" --become --ask-become-pass
BECOME password:
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
192.168.56.103 | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "cmd": "apt-get update",
  "msg": "[Errno 2] No such file or directory: b'apt-get'",
  "rc": 2,
  "stderr": "",
  "stderr_lines": [],
  "stdout": "",
  "stdout_lines": []
}
192.168.56.104 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "cache_update_time": 1787664051,
```

Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations.

I did not notice much difference between the two commands.

Task 2: Writing our First Playbook

With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command `nano install_apache.yml`. This will create a playbook file called `install_apache.yml`. The `.yaml` is the basic standard extension for playbook files.

Run the `yml` file using the command: `ansible-playbook --ask-become-pass install_apache.yml`. Describe the result of this command.

To verify that `apache2` was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.

[illegible]

The unrecognized package will not be installed.

This time, we are going to put additional tasks into our playbook. Edit the `install_apache.yml`. As you can see, we are now adding an additional command, which is the `update_cache`. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.

Save the changes to this file and exit.

Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
PLAY RECAP *****
192.168.56.103 : ok=1  changed=0  unreachable=0  failed=1  skip
ed=0  rescued=0  ignored=0
192.168.56.104 : ok=3  changed=1  unreachable=0  failed=0  skip
ed=0  rescued=0  ignored=0

ndmlot@workstation:~/CPE212_loterte$
```

Based on the results, something was changed in the Ubuntu Server.

Edit again the `install_apache.yml`. This time, we are going to add a PHP support for the apache package we installed earlier.

Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [Update Repository Index] *****
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [192.168.56.103]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Errno 2] No such file or directory: b'apt-get'", "rc": 2, "stderr": "", "stdout_lines": [], "stdout": "", "stdout_lines": []}
changed: [192.168.56.104]

TASK [Apache Install] *****
ok: [192.168.56.104]

TASK [Add PHP support for Apache] *****
changed: [192.168.56.104]

PLAY RECAP *****
192.168.56.103 : ok=1  changed=0  unreachable=0  failed=1  skip
ed=0  rescued=0  ignored=0
192.168.56.104 : ok=4  changed=2  unreachable=0  failed=0  skip
ed=0  rescued=0  ignored=0

ndmlot@workstation:~/CPE212_loterte$
```

The PHP support installation was successful.

Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'mdmlot@workstation.(none)')
mdmlot@workstation:~/CPE212_loterte$ git config --global user.email "qmdmlotert
@tip.edu.ph"
mdmlot@workstation:~/CPE212_loterte$ git config --global user.name "Mark David
oterte"
mdmlot@workstation:~/CPE212_loterte$ git commit -m "Hands On Activity 4"
[main 8302026] Hands On Activity 4
 1 file changed, 15 insertions(+)
 create mode 100644 install_apache.yml
mdmlot@workstation:~/CPE212_loterte$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 455 bytes | 455.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:qmdmlot/CPE212_loterte.git
   fe495f9..8302026  main -> main
mdmlot@workstation:~/CPE212_loterte$
```

https://github.com/qmdmlot/CPE212_loterte

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

Playbooks allow for the usage of YAML files to do complex infrastructure management and administrative tasks across multiple different remote end devices. Being text based files, they can also be tracked, shared, and stored via Git repositories for consistency across team environments.

2. Summarize what we have done on this activity.

In this activity, I have used ad hoc commands alongside privilege elevation to run apt commands on remote servers. Afterwards, I utilized a playbook to run these package verification and installation commands and stored them in my Github repository for future reference.

Conclusions/Learnings:

I conclude that privilege escalation is required to actually manage and configure the packages of remote servers via Ansible. When dealing with different operating systems, YAML files are a must to actually do consistent package verification and installation such as using apt for Debian operating systems and dnf for CentOS and other RHEL based operating systems.



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.